MACHINE LEARNING ALGORITHMS

A comprehensive, beginner-friendly study guide

Concepts • Algorithm families • Core intuition • Key equations • Evaluation • Practical selection

Who this guide is for

Students and aspiring machine-learning practitioners who want a structured reference to the main algorithms, how they work, when to use them, and how to evaluate their results.

This guide is broad rather than literally exhaustive: research produces new algorithms and variants continuously. It covers the major classical algorithms and essential modern concepts.

Prepared October 2026

Contents

- 1. What machine learning is
- 2. Main learning paradigms
- 3. The machine-learning workflow
- 4. Data preparation and feature engineering
- 5. Supervised learning algorithms
- 6. Unsupervised learning algorithms
- 7. Semi-supervised, self-supervised and reinforcement learning
- 8. Neural networks and deep learning
- 9. Model evaluation metrics
- 10. Overfitting, underfitting and regularization
- 11. Choosing an algorithm
- 12. Common mistakes and responsible ML
- 13. Mini worked examples
- 14. Glossary and learning roadmap

How to use this guide

Read Sections 1–4 first to understand the shared foundations. Then study the algorithm families in Sections 5–8. Use the metrics and selection sections whenever you build a model. For every algorithm, learn its assumptions, strengths, weaknesses, important hyperparameters, and the kinds of data on which it tends to work well.

1. What machine learning is

Machine learning (ML) is a field of artificial intelligence in which a computer system learns patterns from data to make predictions, discover structure, or choose actions. Instead of manually writing every rule, a developer selects a learning method and provides examples, objectives, and feedback.

Core terms

Term	Meaning
Dataset	A collection of examples used to develop or evaluate a model.
Sample / instance	One row, record, image, document, event, or other example.
Feature (X)	An input variable used by the model, such as age, price, or pixel values.
Target / label (y)	The outcome to predict in supervised learning.
Algorithm	The learning procedure that fits a model from data.
Model	The fitted function or structure used to make predictions.
Parameters	Values learned from the training data, such as tree splits or neural-network weights.
Hyperparameters	Settings chosen before or during training, such as tree depth or learning rate.
Training	The process of fitting a model to data.
Inference	Using a trained model to make predictions on new inputs.

A simple mathematical view

A model learns a function that maps inputs to outputs. For supervised learning, the training data contain pairs of inputs and targets.

```
Prediction: ■ = fθ(X)
```

Here, X is the input, ■ is the prediction, and θ represents the learned parameters. Training usually chooses θ to minimize a loss function that measures prediction error.

```
θ* = arg minθ [ average training loss + optional regularization ]
```

The two main prediction task types

- **Regression:** predict a numeric quantity, such as house price, temperature, or delivery time.
- **Classification:** predict a category, such as spam/not spam, disease class, or product type.
- **Other tasks:** clustering, dimensionality reduction, anomaly detection, ranking, recommendation, generation, and sequential decision-making.

AI, ML and deep learning

Artificial intelligence is the broad area of building systems that perform tasks associated with intelligence. Machine learning is one approach within AI. Deep learning is a family of ML methods based on neural networks with multiple layers.

2. Main learning paradigms

Paradigm	Training signal	Typical tasks	Example
Supervised learning	Labeled input-output examples	Regression, classification	Predict whether a transaction is fraudulent
Unsupervised learning	Inputs without target labels	Clustering, compression, structure discovery	Group customers by behavior
Semi-supervised learning	A small labeled set plus many unlabeled examples	Classification with limited labels	Classify images when labeling is expensive
Self-supervised learning	Targets constructed from the data itself	Representation learning, pretraining	Predict masked words or image patches
Reinforcement learning	Rewards or penalties from actions	Sequential decisions and control	Train an agent to navigate a game

Supervised learning

The algorithm sees examples with known answers and learns a relationship between features and targets. Training examples are used to fit the model; validation data guide model selection; test data provide a final, relatively unbiased estimate.

Unsupervised learning

The data have no target labels. The algorithm looks for structure, such as groups, low-dimensional representations, or unusual observations. Results often require domain interpretation; a cluster is not automatically a meaningful real-world category.

Semi-supervised and self-supervised learning

Semi-supervised learning uses both labeled and unlabeled data. Self-supervised learning creates a learning signal from the input itself, such as predicting a hidden part of a sentence. Self-supervised pretraining is important in modern language and vision systems.

Reinforcement learning

An agent observes a state, chooses an action, receives a reward, and moves to another state. It seeks a policy that maximizes long-term cumulative reward. Exploration discovers options; exploitation uses what the agent has learned.

```
Return: Gt = rt+1 + γrt+2 + γ²rt+3 + ...
```

γ (gamma) is a discount factor between 0 and 1 that controls how strongly future rewards count.

3. The machine-learning workflow

1. Define the problem

Specify the decision to support, the target, who uses the output, and what a successful model means.

2. Collect and understand data

Check data source, coverage, units, missing values, label quality, sampling bias, privacy, and legal permissions.

3. Split the data

Create training, validation, and test sets. Split by person, customer, location, or time when observations are related.

4. Establish a baseline

Use a simple rule or model. A complex model is useful only if it improves on a sensible baseline.

5. Prepare features

Clean data, encode categories, scale when needed, and handle missing values using transformations learned on training data only.

6. Train candidate models

Fit a small set of algorithms suited to the task and data size.

7. Tune hyperparameters

Use validation data or cross-validation; avoid tuning repeatedly against the final test set.

8. Evaluate errors

Use task-appropriate metrics, confusion matrices, residual plots, calibration checks, and subgroup analysis.

9. Deploy and monitor

Track latency, cost, data drift, model quality, failure cases, and feedback after release.

10. Retrain or retire

Update only when justified by new data and evaluation; retain rollback and human escalation paths.

Data splitting

Split	Purpose	Common use
Training set	Fit parameters	Often 60–80% of data, depending on data size and design
Validation set	Choose model and hyperparameters	Often 10–20%
Test set	Final evaluation after choices are complete	Often 10–20%

These percentages are only starting points. Small datasets may need cross-validation. Time-series problems usually require chronological splits rather than random shuffling.

Cross-validation

In k-fold cross-validation, split the training data into k folds. Fit the model k times, each time holding one fold out for validation and training on the others. Average the validation scores. Use stratified folds for imbalanced classification when appropriate.

```
CV score = (1/k) × Σ score on held-out fold i
```

Prevent data leakage

Data leakage happens when information unavailable at prediction time influences training or evaluation. Examples include scaling before splitting, using future information to predict the past, duplicate records across train and test, or including a feature derived from the target. Leakage can produce impressive but unrealistic scores.

4. Data preparation and feature engineering

Missing values

- Numerical features: impute with training-set median or mean, or use a model that handles missingness.
- Categorical features: impute with the most frequent category or a dedicated “unknown” category.
- Missingness itself can be informative; add an indicator only when justified and validated.

Categorical encoding

- **One-hot encoding:** creates binary columns for categories; often good for nominal categories.
- **Ordinal encoding:** assigns ordered numbers; use only when the order is meaningful or the model handles categories appropriately.
- **Target encoding:** uses target statistics; can leak labels unless carefully cross-fitted and regularized.

Scaling

Standardization transforms a feature to have approximately zero mean and unit standard deviation. Min-max scaling maps values to a chosen range. Scaling is important for distance-based algorithms, SVMs, and many optimization-based models; tree-based methods usually do not require it.

```
Standard score: z = (x − μ) / σ
```

Feature engineering

- Create meaningful variables from dates, text, domain knowledge, or interactions.
- Transform skewed values, such as using a logarithm for a positive long-tailed feature when appropriate.
- Use domain knowledge to remove identifiers or features that encode the answer rather than a generalizable signal.
- For text, representations include bag-of-words, TF-IDF, and learned embeddings.

Class imbalance

When one class is rare, accuracy can look high even if the model misses most positive cases. Consider precision, recall, PR-AUC, class weights, threshold selection, and resampling within training folds. Never resample the full dataset before splitting.

Feature selection and dimensionality

Feature selection chooses a subset of original features. Dimensionality reduction transforms many features into fewer components. These can reduce noise, cost, or overfitting, but may also discard useful information.

5. Supervised learning algorithms

5.1 Linear regression

Purpose: predict a continuous numeric target. It models the target as a weighted sum of features.

```
■ = β■ + β■x■ + β■x■ + ... + βpxp
```

How it learns: ordinary least squares chooses coefficients that minimize the sum of squared residuals.

Strengths: fast, interpretable baseline; coefficient effects are easy to inspect. **Limitations:** a plain linear model cannot represent complex nonlinear patterns unless features are transformed or interactions are added.

Useful assumptions for classical inference: linear relationship in parameters, independent errors, constant error variance, and appropriate error-distribution assumptions for certain tests. Prediction can still be useful when some assumptions are imperfect.

Key metrics: MAE, MSE/RMSE, R^2. **Hyperparameters:** mostly feature choices and regularization strength for regularized variants.

5.2 Logistic regression

Purpose: classification, often binary. Despite its name, it predicts class probabilities.

```
p(y=1|x) = 1 / (1 + e^(−z)); z = β■ + β■x
```

The logistic function maps a linear score to a probability between 0 and 1. A threshold, often 0.5 but not always optimal, converts probability into a class.

Strengths: efficient, interpretable, good baseline, supports probability estimates. **Limitations:** linear decision boundary unless nonlinear features are added. **Key settings:** regularization type and strength, class weights, threshold.

5.3 k-Nearest Neighbors (k-NN)

For a new sample, find the k closest training samples. Classification uses a vote or weighted vote; regression averages their target values.

```
Euclidean distance: d(x,z) = √Σj (xj − zj)²
```

Strengths: simple, non-parametric, flexible. **Limitations:** prediction can be slow, scaling matters, and performance degrades in high dimensions. **Key settings:** k, distance metric, weighting, feature scaling.

5.4 Decision tree

A tree recursively splits the feature space into regions. Each split is chosen to improve a criterion such as impurity reduction for classification or squared error reduction for regression.

Strengths: captures nonlinearities and interactions, handles mixed feature types with suitable implementations, easy to visualize when small. **Limitations:** deep trees overfit and can be unstable. **Key settings:** maximum depth, minimum samples per leaf/split, pruning, split criterion.

Classification impurity examples:

```
Gini = 1 − Σk pk² | Entropy = −Σk pk log■(pk)
```

5.5 Random forest

A random forest trains many decision trees on bootstrap samples and random subsets of features. It averages regression predictions or combines classification votes/probabilities.

Strengths: strong general-purpose baseline, handles nonlinear patterns, usually less variable than a single tree. **Limitations:** less interpretable, can be large, and may not extrapolate well. **Key settings:** number of trees, max features, depth, leaf size, class weights.

5.6 Gradient boosting

Boosting builds models sequentially, with each new model focusing on errors or residual structure left by the existing ensemble. Gradient boosting optimizes a differentiable loss; popular implementations include XGBoost, LightGBM, and CatBoost.

Strengths: often excellent on structured/tabular data. **Limitations:** requires careful tuning and can overfit noisy data. **Key settings:** learning rate, number of trees, tree depth/leaves, row/feature subsampling, regularization.

A low learning rate with more boosting rounds can improve generalization, but requires validation and more compute.

5.7 Support Vector Machine (SVM)

SVM finds a decision boundary with a large margin between classes. Kernel functions allow nonlinear boundaries without explicitly creating all transformed features.

Strengths: effective in some high-dimensional spaces; kernel SVMs model nonlinear boundaries. **Limitations:** scaling is important; kernel SVMs can be expensive on large datasets and probability estimates need care. **Key settings:** C, kernel, gamma, class weights.

```
Soft-margin idea: balance margin size against penalty for training violations
```

5.8 Naive Bayes

Naive Bayes applies Bayes' theorem while assuming features are conditionally independent given the class. Variants include Gaussian, Multinomial, and Bernoulli Naive Bayes.

```
P(class | features) ∝ P(class) × Πj P(feature j | class)
```

Strengths: very fast, works well for some text classification tasks and small datasets. **Limitations:** independence is often unrealistic; probability estimates can be poorly calibrated.

5.9 Linear and quadratic discriminant analysis (LDA/QDA)

LDA models each class with a Gaussian distribution and a shared covariance matrix, leading to linear boundaries. QDA allows a separate covariance matrix per class, giving quadratic boundaries.

Strengths: efficient, useful with suitable distributional structure. **Limitations:** assumptions may not fit real data; QDA needs more data to estimate more parameters.

5.10 Generalized linear models

Generalized linear models extend linear predictors to targets such as counts or binary outcomes using a link function and a probability distribution. Examples include Poisson regression for counts and logistic regression for binary labels. Negative binomial regression can help with overdispersed counts.

5.11 Regularized linear models: Ridge, Lasso and Elastic Net

Method	Penalty	Typical effect
Ridge	$\lambda\Sigma\beta_j^2$ (L2)	Shrinks coefficients; usually keeps all features
Lasso	$\lambda\Sigma\lvert\beta_j\rvert$ (L1)	Can set some coefficients exactly to zero
Elastic Net	Combination of L1 and L2	Balances sparsity and grouped-feature stability

Regularization penalizes model complexity to reduce overfitting. Standardize features before comparing coefficient penalties, and select λ using validation or cross-validation.

5.12 Other supervised methods

- **Polynomial regression:** linear regression on polynomial feature expansions; can model smooth nonlinear relationships but high degrees may overfit.

- **Partial least squares (PLS):** creates components that relate predictors to targets, useful with correlated high-dimensional features.
- **Generalized additive models (GAMs):** add smooth functions of individual features for flexible but partly interpretable relationships.
- **Rule-based learners:** learn or apply human-readable rules; useful where auditability is a priority.
- **Learning to rank:** optimizes the ordering of items, used in search and recommendation systems.

6. Unsupervised learning algorithms

6.1 K-means clustering

K-means partitions observations into k clusters by alternating between assigning each point to its nearest centroid and updating each centroid to the mean of its assigned points.

```
Objective: minimize Σi ||xi − μc(i)||²
```

Strengths: fast and simple for compact, roughly spherical clusters. **Limitations:** must choose k; sensitive to scale, outliers, and initialization; poor for irregular cluster shapes. Use multiple initializations and inspect results.

6.2 Hierarchical clustering

Agglomerative clustering starts with individual observations and repeatedly merges clusters according to a linkage rule. The dendrogram shows the merge structure; cut it at a chosen level to form clusters.

Linkage: single (minimum distance), complete (maximum distance), average, or Ward's method. Useful when the hierarchy itself is informative, but memory and runtime can be expensive for large datasets.

6.3 DBSCAN

DBSCAN forms clusters from dense regions and labels low-density points as noise. It uses a neighborhood radius (eps) and a minimum number of points (min_samples).

Strengths: finds irregular shapes and flags noise; no need to specify cluster count. **Limitations:** parameters are sensitive; variable-density clusters and high-dimensional data can be difficult.

6.4 Gaussian mixture models (GMM)

A GMM models the data as a weighted mixture of Gaussian distributions. The Expectation-Maximization (EM) algorithm alternates between estimating each point's probability of belonging to components (E-step) and updating component parameters (M-step).

Unlike hard clustering, GMM gives soft membership probabilities. Select component count with validation, AIC/BIC, and domain sense.

6.5 Principal Component Analysis (PCA)

PCA transforms correlated numeric features into orthogonal directions called principal components. The first component captures the largest possible variance; each later component captures the largest remaining variance subject to orthogonality.

Uses: compression, visualization, noise reduction, and preprocessing. **Limitations:** components can be hard to interpret; high variance is not necessarily predictive. Scale features when units differ substantially.

6.6 t-SNE and UMAP

These nonlinear dimensionality-reduction methods are often used to visualize high-dimensional embeddings in two or three dimensions. They aim to preserve selected neighborhood relationships, not every global distance. Visual clusters can change with settings and should not be treated as proof of natural categories.

6.7 Association rules

Association-rule mining finds co-occurrence patterns such as "customers who buy A often buy B." Apriori and FP-Growth are common methods.

```
Support(A→B) = P(A∩B) | Confidence = P(B|A) | Lift = P(B|A)/P(B)
```

Lift greater than 1 indicates positive association relative to independence, but does not establish causation.

6.8 Anomaly detection

Anomaly detection flags unusual observations. Methods include Isolation Forest, Local Outlier Factor (LOF), one-class SVM, robust statistical rules, and autoencoders.

- **Isolation Forest:** anomalies are isolated with fewer random splits on average.
- **LOF:** compares local density of a point with the density of its neighbors.
- **One-class SVM:** learns a boundary around typical examples.

An anomaly is not automatically an error or fraud. Set thresholds based on review capacity and the cost of false positives and false negatives.

7. Semi-supervised, self-supervised and reinforcement learning

Semi-supervised methods

- **Pseudo-labeling:** use a trained model to label confident unlabeled examples, then retrain carefully.
- **Consistency regularization:** encourage predictions to remain stable under reasonable input perturbations.
- **Graph-based methods:** propagate information over a similarity graph under assumptions about connected examples.

These methods can help when labels are expensive, but incorrect pseudo-labels can reinforce model mistakes.

Self-supervised learning

Self-supervised learning constructs a pretext task from unlabeled data. Language models may predict masked tokens or the next token; vision models may learn from augmented views or predict missing patches. The resulting representations can be fine-tuned for downstream tasks.

Reinforcement learning: main algorithms

Algorithm	Core idea	Common setting
Q-learning	Learn action values using a temporal-difference target; off-policy	Small/discrete state-action problems
SARSA	On-policy temporal-difference learning	Problems where behavior policy matters
Deep Q-Network (DQN)	Neural network estimates Q-values; replay buffer and target network stabilize learning	Discrete action spaces
Policy gradient	Directly optimize a parameterized policy using reward feedback	Stochastic or continuous policies
Actor–critic	Actor selects actions; critic estimates value or advantage	Many practical RL tasks
PPO	Policy optimization with a clipped update objective	Widely used on-policy deep RL
Model-based RL	Learn or use a model of environment dynamics for planning	When planning or sample efficiency matters

```
Q-learning target: r + γ maxa' Q(s', a')
```

RL is often data-hungry and sensitive to reward design, exploration, simulation fidelity, and random seeds. Evaluate across multiple runs and prevent unsafe real-world exploration.

8. Neural networks and deep learning

Basic neural network

A neural network composes layers of weighted transformations and nonlinear activation functions. Training usually uses backpropagation to calculate gradients and an optimizer such as stochastic gradient descent or Adam to update weights.

```
Layer: h = activation(Wx + b)
```

Common activations include ReLU, sigmoid, and tanh. Softmax is commonly used to turn class logits into a probability distribution.

Core architectures

Architecture	What it is suited to	Important caveat
Multilayer perceptron (MLP)	Tabular baselines and general vector inputs	Needs suitable preprocessing; can overfit small data
Convolutional neural network (CNN)	Images and local spatial patterns	Architecture and resolution matter
Recurrent neural network (RNN/LSTM/GRU)	Sequential signals and older sequence pipelines	Long-range dependencies can be challenging
Transformer	Text, vision, audio and multimodal sequence modeling	Attention can be compute- and memory-intensive
Autoencoder	Learn compressed representations; anomaly detection in some settings	Reconstruction error is not automatically an anomaly score
Generative adversarial network (GAN)	Generate samples through generator–discriminator competition	Training can be unstable
Diffusion model	Generate samples by learning to reverse a noise process	Sampling can be computationally expensive

Training concepts

- **Loss function:** measures mismatch between prediction and target, such as cross-entropy for classification or squared error for regression.
- **Backpropagation:** computes gradients of the loss with respect to model parameters.
- **Learning rate:** controls update size; too high can destabilize training, too low can make learning slow.
- **Batch size:** number of examples processed per update.
- **Epoch:** one pass through the training dataset.
- **Dropout:** randomly disables activations during training as a regularization method.
- **Early stopping:** stop when validation performance stops improving.
- **Transfer learning:** start from a pretrained model and adapt it to a new task.

Embeddings

An embedding maps an object such as a word, sentence, image, or user into a numeric vector. Useful embeddings place objects with similar learned characteristics near one another under a chosen similarity metric. Embeddings support semantic search, recommendation, clustering, and retrieval-augmented generation (RAG).

When deep learning is appropriate

Deep learning is especially useful for large-scale unstructured data such as images, audio, and text, and when pretrained models are available. For modest tabular datasets, gradient-boosted trees and regularized linear models are often strong first choices.

9. Model evaluation metrics

Regression metrics

Metric	Definition / interpretation	Caution
MAE	Mean of |y – ■|; average absolute error in target units	Treats errors linearly
MSE	Mean of (y – ■)²; heavily penalizes large errors	Units are squared
RMSE	Square root of MSE; error in target units	Sensitive to outliers
R^2	1 – residual sum of squares / total sum of squares	Can be negative on test data; not a percent-correct score
MAPE	Mean absolute percentage error	Unstable or undefined when true values are zero or near zero

```
MAE = (1/n)Σ|yi – ■i| RMSE = √[(1/n)Σ(yi – ■i)²]
```

Classification metrics

Metric	Definition / interpretation	Best used when
Accuracy	(TP + TN) / all predictions	Classes are reasonably balanced and error costs are similar
Precision	TP / (TP + FP)	False positives are costly
Recall / sensitivity	TP / (TP + FN)	Missing positives is costly
Specificity	TN / (TN + FP)	Correctly rejecting negatives matters
F1	Harmonic mean of precision and recall	Balance precision and recall; positive class is important
ROC-AUC	Ranking quality across thresholds using true-positive and false-positive rates	Compare binary ranking performance
PR-AUC	Precision–recall curve area	Often more informative with rare positive classes
Log loss	Penalizes incorrect probabilities, especially confident errors	Probability quality matters
Balanced accuracy	Average recall across classes	Class imbalance

```
Precision = TP/(TP+FP) Recall = TP/(TP+FN) F1 = 2PR/(P+R)
```

TP = true positive; TN = true negative; FP = false positive; FN = false negative. When a denominator is zero, software conventions may define the metric as zero, undefined, or another documented value.

Confusion matrix

	Predicted positive	Predicted negative
Actual positive	TP	FN
Actual negative	FP	TN

Threshold selection and calibration

A model may output a probability, but the operating threshold should reflect the costs of errors, class prevalence, and available review capacity. A calibrated model's predicted probability of 0.8 should correspond to roughly 80% positives among comparable predictions. Calibration is different from discrimination/ranking.

Clustering metrics

- **Silhouette score:** compares within-cluster distance with distance to the nearest alternative cluster; higher is generally better, but shape assumptions matter.
- **Davies–Bouldin index:** lower is generally better.
- **Adjusted Rand Index (ARI):** compares clusters with known labels when labels are available for evaluation; labels are for evaluation, not necessarily training.

No single clustering metric proves that clusters are useful. Assess stability and domain meaning.

10. Overfitting, underfitting and regularization

Pattern	Training performance	Validation performance	Likely response
Underfitting	Poor	Poor	Use a more expressive model, improve features, reduce excessive regularization
Good generalization	Good	Similar and good	Confirm with a final test set and monitor in production
Overfitting	Very good	Much worse	Simplify model, regularize, collect more data, improve validation design

Bias–variance trade-off

High bias means the model is too constrained to capture important structure. High variance means small changes in training data produce large changes in the fitted model. The goal is not to make the training error zero; it is to generalize to unseen cases.

Regularization strategies

- L1/L2 penalties on parameters.
- Limit tree depth or increase minimum leaf size.
- Dropout, data augmentation, and weight decay for neural networks.
- Early stopping based on validation performance.
- More representative training data, careful feature selection, and simpler models.

Ensemble learning

An ensemble combines multiple models. Bagging (for example, random forests) trains models in parallel on resampled data to reduce variance. Boosting trains models sequentially to correct errors. Stacking trains a meta-model on out-of-fold predictions. Ensembles can improve accuracy but increase complexity and operational cost.

Data drift and concept drift

Data drift means the input distribution changes. Concept drift means the relationship between inputs and targets changes. Monitor data quality, input distributions, delayed labels, and subgroup performance. A drift alarm does not automatically mean retraining is necessary; investigate impact first.

11. Choosing an algorithm

Situation	Good starting candidates	Reason / checks
Numeric target, modest tabular data	Linear/Ridge/Lasso; Random Forest; Gradient Boosting	Compare a simple interpretable baseline with nonlinear models
Binary classification	Logistic Regression; Random Forest; Gradient Boosting	Choose metrics and threshold based on error costs
High-dimensional sparse text	Naive Bayes; Logistic Regression with TF-IDF; linear SVM	Strong efficient baselines
Image, audio or raw text	Pretrained deep model / transfer learning	Check data volume, compute, licensing, and domain match
Customer segmentation	K-means; hierarchical; GMM; DBSCAN	Scale features and validate whether clusters are actionable
Outlier detection	Isolation Forest; LOF; robust statistics	Set threshold based on investigation capacity
Compress numeric features	PCA; autoencoder	Check reconstruction and downstream task performance
Time-series forecasting	Naive seasonal baseline; regression with lags; dedicated forecasting models	Use chronological validation and avoid future leakage
Sequential actions and rewards	Bandits or reinforcement learning	Consider simpler contextual bandits before full RL

A practical selection checklist

- What exactly is the target, and what information is available at prediction time?
- How many examples and features exist? Are labels reliable?
- Are data tabular, text, image, audio, time-series, graph, or mixed?
- Do you need explanations, calibrated probabilities, low latency, or low compute cost?
- What are the costs of false positives and false negatives?
- What is the baseline, and does the candidate beat it on held-out data?
- Will the model remain reliable across groups, time periods, and real deployment conditions?

Practical rule: start with a simple baseline, use a leakage-safe validation process, compare a small number of suitable models, and choose the simplest model that meets the requirements.

12. Common mistakes and responsible ML

Technical mistakes

- Evaluating on training data and calling the result generalization performance.
- Tuning repeatedly on the test set until it is no longer an independent final check.
- Preprocessing the full dataset before splitting, which leaks information.
- Using accuracy alone for imbalanced data.
- Ignoring uncertainty, confidence intervals, and variation across random seeds.
- Assuming feature importance proves causation.
- Using a random split for time-dependent data or for related records that should stay together.
- Deploying without monitoring, rollback, or a plan for missing/invalid inputs.

Responsible ML

A model can reproduce historical inequities, fail on underrepresented groups, expose sensitive data, or be misused. Responsible development includes data minimization, consent and lawful use, privacy and security controls, subgroup evaluation, explainability appropriate to the stakes, human oversight, and documentation of limitations.

Interpretability tools

- **Coefficients:** inspect effects in linear models, while considering scaling and correlated features.
- **Permutation importance:** measure performance change when one feature is shuffled; correlated features can complicate interpretation.
- **Partial dependence / ICE:** explore how predictions vary with a feature, with care when features are correlated.
- **SHAP-style explanations:** attribute predictions to features under a defined attribution framework; explanations are not proof of causality.

Causal inference is different

Prediction asks what is likely to happen. Causal inference asks what would happen under an intervention. High predictive accuracy alone does not establish that changing a feature will cause the predicted outcome.

13. Mini worked examples

Example A: classify spam messages

Task: predict spam vs. not spam from message text. **Baseline:** predict the majority class. **Features:** TF-IDF text vectors. **Models:** Multinomial Naive Bayes, logistic regression, linear SVM. **Evaluation:** precision, recall, F1, PR-AUC if spam is rare. **Deployment:** monitor false positives and allow user reporting.

Example B: predict house prices

Task: estimate sale price from size, location, age, and other attributes. **Baseline:** median training-set price. **Models:** Ridge regression, random forest, gradient boosting. **Evaluation:** MAE and RMSE, with error analysis by price range and location. **Leakage warning:** do not use information only available after the sale.

Example C: segment customers

Task: discover behavioral groups without known labels. **Preparation:** select meaningful behavior variables and scale where needed. **Models:** K-means, hierarchical clustering, or DBSCAN. **Evaluation:** silhouette/stability plus business interpretation. **Warning:** clusters depend on feature choices and do not automatically imply distinct customer types.

Example D: document question answering

Task: answer questions using a document collection. **Typical pipeline:** extract text, split into chunks, create embeddings, retrieve relevant chunks, and give those chunks to a language model to generate an answer. Retrieval quality can be assessed with recall@k or ranking metrics; answer quality needs groundedness and correctness checks. This is a system combining retrieval and a generative model, not a single classical ML algorithm.

Example E: speech recognition

Task: convert speech audio into text or classify an accent. **Inputs:** waveform or acoustic features/representations. **Models:** traditional acoustic models, CNN/RNN systems, Transformers, or pretrained speech models. **Evaluation:** word error rate for transcription; accuracy, macro-F1, and subgroup analysis for accent classification. Control for speaker overlap between training and test sets.

14. Glossary and learning roadmap

Quick glossary

Term	Plain-language definition
Loss	A numeric measure of how wrong predictions are under a chosen objective.
Gradient descent	An iterative method that updates parameters in a direction that reduces loss.
Learning rate	The size of a parameter update.
Regularization	A constraint or penalty that discourages overly complex fits.
Hyperparameter tuning	Selecting training settings using validation data.
Feature importance	A measure of how much a feature contributes to a model or its predictions under a specific method.
Inference	Generating predictions with a fitted model.
Generalization	Performance on new data drawn from the intended use setting.
Calibration	Agreement between predicted probabilities and observed frequencies.
Drift	A change in data distributions or the target relationship over time.
Embedding	A learned vector representation of an item.
Baseline	A simple reference method used to judge whether a more complex model adds value.

Suggested learning roadmap

Stage 1 — Foundations

Python, NumPy, pandas, visualization, descriptive statistics, probability, vectors/matrices, derivatives, and train/validation/test splits.

Stage 2 — Classical supervised ML

Linear regression, logistic regression, k-NN, decision trees, random forests, gradient boosting, SVM, and Naive Bayes.

Stage 3 — Evaluation

Metrics, cross-validation, leakage, class imbalance, calibration, error analysis, and feature engineering.

Stage 4 — Unsupervised ML

K-means, hierarchical clustering, DBSCAN, PCA, GMM, and anomaly detection.

Stage 5 — Practical projects

Build end-to-end projects with reproducible preprocessing, baselines, model comparison, clear metrics, and a README explaining limitations.

Stage 6 — Deep learning

Neural-network fundamentals, PyTorch or TensorFlow, CNNs, Transformers, transfer learning, and embeddings.

Stage 7 — Deployment

Serve a model with FastAPI, validate inputs, log safe operational metrics, monitor drift, and document retraining criteria.

Recommended practice routine

- For each algorithm, implement or use it on a small dataset, explain its assumptions, tune 2–4 key hyperparameters, and compare it with a baseline.
- Write down the metric before training, then inspect errors rather than focusing only on a single score.
- Keep a record of dataset versions, random seeds, preprocessing, model settings, and results.

Final takeaway

There is no universally best machine-learning algorithm. The best choice depends on the task, data quality and size, target, evaluation design, interpretability needs, costs, and deployment environment. Strong ML work is a disciplined process of problem definition, leakage-safe experiments, honest evaluation, and ongoing monitoring—not simply choosing the most complex model.

Further study: reliable starting points

Use official documentation and textbooks to deepen the topics in this guide. Versions and APIs change, so check the current documentation for implementation details.

- Scikit-learn User Guide — classical ML, preprocessing, model selection, metrics, and pipelines: https://scikit-learn.org/stable/user_guide.html
- PyTorch Tutorials — neural networks and deep learning workflows: https://pytorch.org/tutorials/
- TensorFlow Tutorials — deep learning and model deployment: https://www.tensorflow.org/tutorials
- Google Machine Learning Crash Course — practical introductory lessons: https://developers.google.com/machine-learning/crash-course
- An Introduction to Statistical Learning — accessible statistical learning textbook: https://www.statlearning.com/
- The Elements of Statistical Learning — deeper statistical foundations: https://hastie.su.domains/ElemStatLearn/

Note: URLs are included as plain text for reference. This guide is educational and does not replace task-specific validation or expert review for high-stakes applications.